

```

// CreateTodo.telefunc.ts
// Environment: server

// Telefunc makes onNewTodo() remotely callable
// from the browser.
export { onNewTodo }

import { getContext } from 'telefunc'

// Telefunction arguments are automatically validated
// at runtime, so `text` is guaranteed to be a string.
async function onNewTodo(text: string) {
  const { user } = getContext()

  // With an ORM
  await Todo.create({ text, authorId: user.id })

  // With SQL
  await sql(
    'INSERT INTO todo_items VALUES (:text, :authorId)',
    { text, authorId: user.id }
  )
}

```

Server

```

// CreateTodo.tsx
// Environment: client

// CreateTodo.telefunc.ts isn't actually loaded;
// Telefunc transforms it into a thin HTTP client.
import { onNewTodo } from './CreateTodo.telefunc.ts'

async function onClick(form) {
  const text = form.input.value
  // Behind the scenes, Telefunc makes an HTTP request
  // to the server.
  await onNewTodo(text)
}

function CreateTodo() {
  return (
    <form>
      <input input="text"></input>
      <button onClick={onClick}>Add To-Do</button>
    </form>
  )
}

```

Browser

Intro
○○○○

Hooks
○○○○○○○

Architecture
○○

Use cases
○○○

DX
○○●○○○○○

Business model
○

Future
○

Conclusion
○